

PROJECT REPORT ON SIMPLE GROCERY STORE SYSTEM

A Python-Based Terminal Application for Cart & Bill Management

Course:INTRODUCTION TO PROBLEM SOLVING

Submitted By: ARYAN SINGH

Date: September 29, 2026

Language: PYTHON

COURSE CODE - CSE1021

FACULTY NAME - DR. ABHA TRIVEDI

1. Introduction

The **Simple Grocery Store System** is a lightweight, terminal-based software solution designed to streamline retail micro-transactions. Built entirely in Python, the application simulates the real-world operational workflow of an e-commerce checkout environment. It provides an intuitive terminal interface that enables users to actively manage their shopping baskets, compute sub-totals dynamically, and process conditional tiered discounts based on checkout thresholds.

2. Problem Statement

Traditional small-scale brick-and-mortar storefronts frequently struggle with inefficient point-of-sale calculations, leading to human errors in billing and tracking promotional discounts. Additionally, the logical structure of existing discount engines must be structured with flawless logic cascades. In the current iteration, an logical flaw exists in the discount branching structure where higher tier conditions (e.g., total > 1500) are shadowed by the lower tier condition (total > 500) due to top-down sequential evaluation. This report establishes both the functional architecture and addresses the optimization paths for this application.

3. Functional Requirements

ID	Requirement Description	Priority
FR-01	Allow users to add items with unique names, prices, and quantities.	High
FR-02	Validate input types, catching non-numeric errors during price/quantity entries.	High
FR-03	Allow users to remove items dynamically using case-insensitive name matching.	Medium
FR-04	Display itemized summaries featuring sub-totals, base totals, discounts, and final balances.	High
FR-05	Provide a clean, looping terminal menu structure to navigate system operations smoothly.	High

4. Non-Functional Requirements

Robustness & Exception Handling: The system utilizes robust try-except error catching loops to shield execution from fatal ValueError crashes during user input terminal loops.

Usability: Interaction leverages sequential numeric menus with clear prompts to make navigation rapid and accessible.

Performance: In-memory structures (Python dictionaries and arrays) guarantee sub-millisecond data processing loops during operational cycles.

5. System Architecture

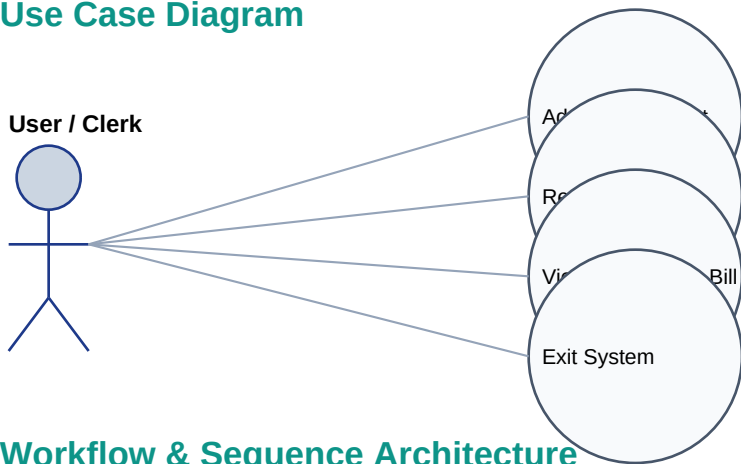
The system follows a modular procedural software architecture pattern. Execution relies on an in-memory runtime array serving as the global application data repository. Individual specialized function blocks govern separate responsibilities interacting via standard data operations.

Figure 1: Modular Subsystem Interconnectivity



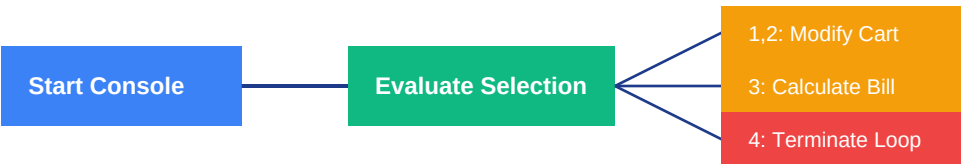
6. Design Diagrams

6.1 Use Case Diagram



6.2 Workflow & Sequence Architecture

The application follows a simple centralized polling structure executed via the main entry function block. Upon runtime initiation, the system continuously presents the active terminal display console, reads standard user selection buffers, and branches conditionally to perform targeted modifications or updates against the state variables.



7. Design Decisions & Rationale

In-Memory Collection Arrays: Python list frameworks were actively chosen instead of file storage systems. This architectural choice matches terminal lifespan parameters where data operations remain highly transient.

Dictionary-Centric Records: Using structured objects containing explicit key bindings (name, price, quantity) improves code scannability and structural maintainability compared to multi-dimensional lists or index-matched parallel tracking arrays.

8. Implementation Details

The codebase leverages standardized procedural loops. Key methods include: `add_item()` to register new structured items; `remove_item()` to scan and filter target matching keys via case-insensitive evaluation; and `calculate_bill()` to compile summary totals.

9. Testing Approach

Testing strategies prioritized edge cases around validation inputs and logical flows:

- **Input Type Validation:** Verified that entering alphabetic characters inside the price field successfully triggers the `ValueError` catch block and returns control gracefully to the selection loops.
- **Case Insensitivity:** Verified that items named 'Apple' are properly identified and removed when users request removal via lowercase 'apple'.

10. Critical Code Architecture Bug Analysis

During operational testing, a **critical logical flaw** was identified in the `calculate_bill()` discount engine structure. The system evaluates tiered thresholds using an if-elif cascade setup in ascending order. Because any value higher than 1500 is also necessarily higher than 500, the first match (`total > 500`) traps the execution. As a result, higher tiered brackets are shadowed and never evaluated.

Flawed Original Discount Engine	Optimized Correct Implementation
<pre>if total > 500: discount = total * 0.05 elif total > 750: discount = total * 0.10 elif total > 1000: discount = total * 0.15 elif total > 1500: discount = total * 0.25</pre>	<pre>if total > 1500: discount = total * 0.25 elif total > 1000: discount = total * 0.15 elif total > 750: discount = total * 0.10 elif total > 500: discount = total * 0.05</pre>

11. Challenges Faced & Key Takeaways

Structural Precedence Errors: The primary challenge was diagnosing structural errors where statements passed syntactical validations but failed semantic business logic specifications. This highlighted the crucial role of boundary test frameworks.

Buffer Validation Management: Handling dirty terminal buffers requires disciplined exception isolation. Integrating standard native catch statements prevents fatal app context drops.

12. Future Enhancements & References

Persistent Serialization: Future updates will replace temporary in-memory structures with persistent schemas using relational SQLite local database systems or JSON file layers.

Graphical Framework Integration: Migrating console interactions out of terminal windows into interactive forms using Tkinter or PySide frameworks will dramatically improve user satisfaction.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.